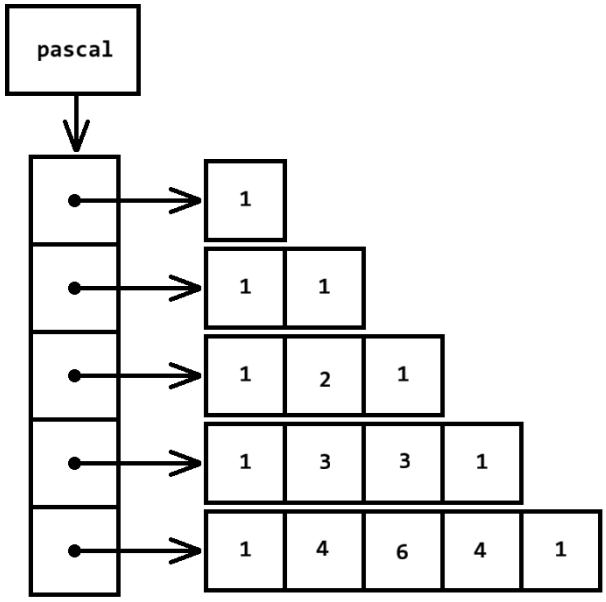
İSTANBUL ÜNİVERSİTESİ-CERRAHPAŞA MÜHENDİSLİK FAKÜLTESİ BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ		
BIMU2057 – Veri Yapıları – Vize		Tarih: 17/11/2025
Öğrenci Numarası:	Sınıf:	Süre: 75 dk
Adı Soyadı:		İmza:

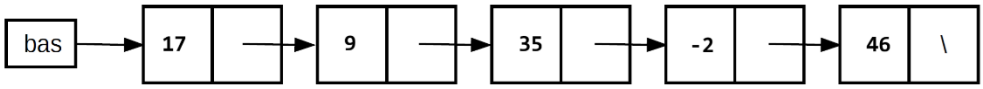
Soru	S1 (20p)	S2 (24p)	S3 (16p)	S4 (40p)	Toplam:
ÖÇ	1, 2	1, 2	1, 2	1, 2, 3	
PÇ	1	1	4	4	
Not:					

1) Binom açılımını dinamik bellek yönetimi ile (`new int*[…]/int[…]`) Pascal üçgeni olarak hesaplayıp pointerın pointerı olarak döndüren `int** generatePascal(int numRows)` fonksiyonunu yazınız. Bu fonksiyonun aşağıdaki şekilde çağırılması sonrası `pascal` değişkeni yandaki şekilde verildiği gibi olmalıdır.

```
int** pascal = generatePascal(5);
```



2)



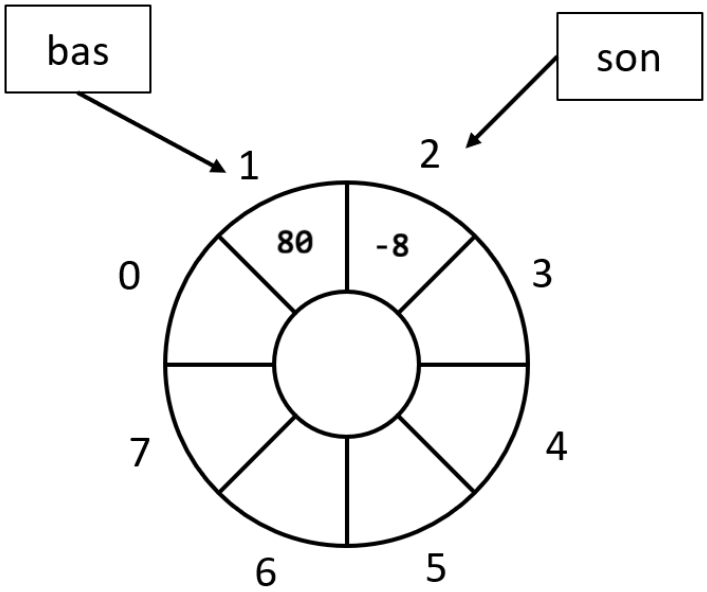
Yukarıdaki bağlı liste üzerinde, yandaki komut satırlarının çalıştırıldığı kabul edilmektedir. İşaretlenen a, b ve c anlarında listenin durumunu çiziniz.

Not: Bağlı liste içerisinde aşağıdaki düğüm yapısının kullanıldığı kabul edilmektedir.

```
struct Dugum{
    int data;
    Dugum* sonraki = nullptr;
    Dugum(int d) : data(d) {}
};
```

```
Dugum *d, *g, *m;
d = head;
g = head->sonraki->sonraki;
m = g->sonraki->sonraki;
m->sonraki = d->sonraki;
head = m->sonraki;
delete d;
// ----- a -----
d = g->sonraki;
g->sonraki = m;
delete d;
// ----- b -----
m = new Dugum(-12);
m->sonraki = g;
head->sonraki = m;
// ----- c -----
```

Ders Öğretim Üyesi:
Dr.Öğr.Üyesi Nihal ALTUNTAŞ




```
for(int i = 0; i < 5; i++) {
    cQueue.enqueue(i+5);
}
for(int i = 0; i < 3; i++) {
    cQueue.dequeue();
}
for(int i = 0; i < 5; i++) {
    cQueue.enqueue(10*i);
}
for(int i = 0; i < 3; i++) {
    cQueue.dequeue();
}
```

3) Yukarıda solda, 8 adet int değeri kapasitesine sahip ve `cQueue` değişkeni ile tutulan döngüsel kuyruk dizisinin mevcut hali verilmektedir. Kuyruğun yukarıda sağdaki komut satırları çalıştırıldıktan sonraki durumunu çiziniz.

4) Yanda bağlı liste ile uygulaması yapılmış bir yığın yapısı verilmektedir. Yığın yapısı içerisinde bulunan fonksiyonları aşağıda verilen açıklamalara göre yazınız.

```
struct Dugum {
    char parantez;
    Dugum *sonraki = nullptr;
    Dugum (char p) : parantez(p) {}
};

struct Yigin {
    Dugum *tepe = nullptr;
    void push(char p);
    char pop();
    bool bosMu();
    bool gecerliMi(char denklem[]);
};
```

- **push:** Parametre olarak aldığı karakter için yeni bir düğüm oluşturarak yığının tepesine ekler.
- **pop:** Yığının tepesindeki karakteri döndürür. Tepe karakteri döndürmeden önce ilgili düğümü hafızadan serbest bırakır. Eğer yığın boş ise '\0' döndürür.
- **bosMu:** Yığının boş olup olmadığı bilgisini döndürür.
- **gecerliMi:** Parametre olarak aldığı denklem içerisindeki (), [] ve {} parantez çiftlerinin düzgün kullanılıp kullanılmadığını kontrol ederek denklemin geçerli olup olmadığını döndürür. Aşağıdaki örnek denklemler için gecerliMi fonksiyonun sonucu yorum satırı olarak verilmiştir.

```
char denklem1[] = "{[(2+3)*7]-(24/6)}";
char denklem2[] = "{[(2+3)*7]-(24/6))";
char denklem3[] = "{[(2+3)*7]-(24/6)";
Yigin y;
// y.gecerliMi(denklem1) → true
// y.gecerliMi(denklem2) → false
// y.gecerliMi(denklem3) → false
```

Ders Öğretim Üyesi:
Dr.Öğr.Üyesi Nihal ALTUNTAŞ